

Universidad Rey Juan Carlos

Escuela Técnica Superior
Ingeniería Informática

Asignatura: Procesadores de Lenguajes

Autores:

Diego Iglesias Peña

Sergio Muñoz Laureiro

Índice

1. Introducción
2. Reglas Léxicas
3. Transformación de la gramática a LL(1)
4. Notificación y recuperación de errores
5. Traducción Dirigida por Sintaxis
6. Explicación de los casos de prueba
7. Anexo: conjuntos directores

1. Introducción

El presente documento detalla el diseño e implementación de los analizadores léxico y sintáctico para un lenguaje de programación de cálculo científico, desarrollado como primera fase del traductor orientado a C. Se ha implementado tanto la funcionalidad obligatoria como las ampliaciones opcionales correspondientes a esta fase (constantes en nuevas bases, constantes lógicas y sentencias de control de flujo ampliadas).

2. Reglas Léxicas

De acuerdo con las especificaciones, el analizador léxico presenta ciertas expresiones regulares que requieren atención especial por su complejidad:

- Constantes literales (Cadenas): Se ha diseñado la regla *STRING_CONST* para soportar delimitadores simples (') y dobles ("). El aspecto más relevante es el soporte para el escape de comillas internas mediante su duplicación (ej. "" dentro de una cadena delimitada por comillas dobles), lo cual requiere un manejo preciso de los grupos de negación en ANTLR.
- Constantes reales: La regla *NUM_REAL_CONST* engloba el formato de punto fijo, el formato exponencial y el mixto. La complejidad se encontraba en hacer opcionales ciertas partes (como el signo del exponente) sin generar ambigüedades con el signo de resta u otras constantes enteras.
- Constantes numéricas opcionales: Para evitar solapamientos léxicos, las bases binaria, octal y hexadecimal se han restringido para que exijan el prefijo estricto (b, o, z) seguido obligatoriamente de comillas simples envolviendo los dígitos válidos para cada base.

3. Transformación de la Gramática a LL(1)

Con el fin de asegurar un análisis sintáctico que sea eficiente, la gramática inicial fue sometida a un proceso de normalización.

3.1. Eliminación de la recursividad por la izquierda

La recursividad por la izquierda es incompatible con los analizadores que trabajan en forma descendente, ya que ocasiona un bucle infinito al intentar expandir el símbolo no terminal antes de procesar cualquier token.

- Ejemplo en la práctica (*sentlist*):

Originalmente, *sentlist* se define de forma recursiva izquierda:

```
sentlist ::= sentlist sent | sent.
```

Se ha transformado mediante la introducción de un símbolo auxiliar (*sentlistp*) para convertirla en recursiva por la derecha:

```
sentlist : sent sentlistp ;
```

```
sentlistp: sent sentlistp | /*Lambda*/;
```

- Ejemplo en expresiones (*exp*):

Se ha aplicado el mismo principio a las expresiones aritméticas, separando los operandos (*factor*) de la continuación de la operación (*expp*):

```
exp:factor exp;
```

```
expp:op factor exp | /*Lambda*/ ;
```

3.2. Factorización por la izquierda

Este proceso se aplica cuando varias producciones de un mismo no terminal comparten un prefijo, lo que impide que el parser decida el camino a seguir con un solo token.

- Ejemplo: La sentencia IF

En algunos lenguajes, el IF puede ser simple o tener un bloque THEN/ELSE:

```
sent ::= ... | "IF" "(" expcond ")" sent | "IF" "("  
expcond ")" "THEN" sentlist "ENDIF" | "IF"  
"(" expcond ")" "THEN" sentlist "ELSE" sentlist "ENDIF"  
| ...
```

Para evitar el conflicto, se ha extraído el prefijo común("IF" "(" expcond ")") :

```
sent: ... | 'IF' '(' expcond ')' sentif | ...  
sentif : sent // IF simple
```

```
      | 'THEN' sentlist sentthen; // IF con bloque, aquí se vuelve a  
aplicar factorización por la izquierda
```

```
sentthen : 'ENDIF'  
          | 'ELSE' sentlist 'ENDIF';
```

Aquí, la decisión sobre el ELSE se pospone hasta haber procesado el cuerpo del THEN.

3.3. Resolución de conflictos y Lookahead

Se han identificado situaciones donde la diferencia semántica requeriría una gramática mucho más complicada para ser estrictamente LL(1). En estas circunstancias se ha optado por:

1. Ambigüedad en Declaraciones (*codproc* y *codfun*): Al definir subprogramas, el token tipo (*INTEGER*, *REAL*, etc.) aparece tanto en la lista de parámetros (*dec_s_paramlist*) como en las declaraciones locales (*dcllist*). Dado que *dec_s_paramlist* puede ser

vacío, el parser ve un tipo y no sabe a qué sección pertenece sin mirar más adelante. Tratamos de corregir esto, pero por la complejidad de este cambio, decidimos dejarlo así, resultando en LL(2). Además, en la regla *codfun* tenemos también un problema parecido con *sentlist* e *IDENT*. En este caso, cuando encuentra *IDENT* no sabe si pertenece a *sentlist* o es el último *IDENT*. Tratamos de implementar una solución a esto con una nueva regla que tratase de evitar el problema, pero no fue posible ya que o bien acabamos en el mismo problema que en primer caso (el de *dcllist*) o bien permitía algunas estructuras que no debían ser permitidas. También se consideró eliminar un tramo ya que pensamos que podía ser una redundancia, pero esto también resultaba en algo no esperado, por lo que decidimos dejarlo en LL(*).

Se confía en la capacidad de ANTLR4 para resolver estos conflictos, manteniendo la gramática legible y fiel al estándar del lenguaje solicitado.

4. Gestión de Errores: Notificación y Recuperación

La robustez de un procesador de lenguajes se evalúa por su habilidad para manejar entradas incorrectas sin interrumpir de manera brusca el análisis. En esta práctica, se han introducido mecanismos específicos para mejorar la interacción con el usuario y la durabilidad del parser.

4.1. Implementación de la Notificación de Errores

Para mejorar la claridad del feedback, se ha creado un Listener personalizado extendiendo la clase `BaseErrorListener` de ANTLR. Este componente permite interceptar las excepciones antes de que se muestren por la salida estándar, eliminando los manejadores por defecto mediante `removeErrorListeners()`. Al detectar una inconsistencia, el sistema produce una notificación completa que incluye:

- Localización exacta: Se informa de la línea y columna precisas donde el token no coincide con las reglas de la gramática.
- Contexto del fallo: El mensaje indica el token inesperado y, en base a los conjuntos de Directores de la gramática LL(1) transformada, sugiere qué tipo de estructura se esperaba en ese punto.

Para ejecutar el caso de prueba se usará el siguiente comando en la terminal:

```
java -jar miPrograma.jar PRUEBAS/CasoIncorrecto.txt
```

-Resultado en terminal(pruebaNotificacionError.for):

```
Analizando archivo: PRUEBAS/pruebaNotificacionError.for...
ERROR SINTÁCTICO en línea 3:0
missing ';' at 'INTEGER'
ERROR SINTÁCTICO en línea 6:9
mismatched input ';' expecting {'(', IDENT, NUM_INT_CONST_B, NUM_INT_CONST_O, NUM_INT_CONST_H, CONST_BOOL, NUM_INT_CONST, NUM_REAL_CONST, STRING_CONST}
Se detectaron errores. No se generará el archivo C.
```

4.2. Resolución de la Recuperación de Errores

Con el objetivo de permitir que el análisis continúe tras detectar un fallo y poder reportar todos los errores del fichero en una sola pasada, se han aplicado las siguientes estrategias de recuperación:

- Sincronización en Modo Pánico: Ante errores complejos en estructuras iterativas como las listas de sentencias (*sentlist*) o declaraciones (*dcllist*), el parser entra en un estado de búsqueda. Utiliza tokens de sincronización (como ; , *END*) para saltar el fragmento de código erróneo y retomar el análisis en un punto seguro.
- Recuperación en Línea (Mismatched Token Exception): El parser es capaz de realizar reparaciones simples sobre la marcha. Si se detecta la falta de un token predecible, como un paréntesis en una expresión condicional o un delimitador, ANTLR intenta realizar una "inserción virtual" o un "borrado" del token sobrante para validar la regla actual y proseguir con la siguiente instrucción sin detener el flujo de ejecución.

Como se podrá observar en los casos de prueba incorrectos del siguiente apartado, el analizador es capaz de detectar y reportar múltiples errores en una sola pasada sin detenerse en el primero. Además, cuando se detecta algún error, ya sea léxico, sintáctico o semántico, el sistema no genera el archivo .c de salida, garantizando que nunca se produzca código incorrecto.

5. Traducción Dirigida por la Sintaxis (TDS)

Esta sección describe la fase de traducción del lenguaje fuente de cálculo científico al lenguaje objeto (C). La traducción se implementa mediante acciones semánticas integradas directamente en la gramática ANTLR (.g4), coordinadas por una clase Traductor encargada de gestionar la generación y estructuración del código final.

5.1. Arquitectura del traductor

La fase de traducción se articula en torno a acciones integradas en la gramática y al soporte de cuatro clases Java auxiliares:

- **Traductor:** Clase central que acumula los diferentes elementos traducidos (directivas #define, prototipos, funciones, procedimientos y el cuerpo del programa principal) para generar el archivo C final con la estructura jerárquica correcta: en primer lugar las directivas #define, seguidas de los prototipos, las implementaciones de los subprogramas y, finalmente, la función void main(void).
- **Contexto:** Componente que gestiona una pila de ámbitos para determinar en todo momento si el flujo de análisis se encuentra dentro de una función (identificando su nombre para el retorno) o de un procedimiento (identificando qué parámetros se pasan por referencia). Su uso es indispensable para traducir de forma correcta las asignaciones al nombre de la función como sentencias return y para aplicar el operador de indirección * a los parámetros OUT/INOUT.
- **Parametro:** Representa un parámetro formal, almacenando su tipo en C, identificador, y los indicadores booleanos de si corresponde a una cadena de caracteres (esCadena) o si se pasa por referencia (esReferencia). El método toCParam() genera la representación sintáctica exacta en C: char nombre[] para cadenas en prototipos y parámetros (sin dimensión), o tipo *nombre para parámetros de salida o entrada/salida.
- **VariableDecl:** Estructura que representa una variable declarada, registrando su tipo, identificador, dimensión (en el caso de tipos CHARACTER con longitud) y su valor de inicialización opcional.

5.2. Traducción de tipos y constantes

Tipos de datos

Se realiza una correspondencia directa y uniforme entre los tipos del lenguaje fuente y las estructuras nativas del lenguaje final:

- **INTEGER:** int
- **REAL:** float
- **CHARACTER:** char
- **CHARACTER(n):** char[n]

Constantes (#define)

Todas las constantes declaradas mediante el atributo **PARAMETER** se agrupan al inicio del archivo de salida C bajo directivas **#define**, independientemente de su posición original en el código fuente. Las constantes literales de tipo cadena se delimitan mediante comillas dobles en C, escapando los caracteres internos necesarios a través de la función **toCLiteral**, la cual además desduplica correctamente las comillas del lenguaje fuente (transformando " en ' y "" en ").

Constantes en bases alternativas (ampliación opcional)

La traducción de los literales numéricos en bases alternativas y de las constantes lógicas se realiza conforme al estándar de C:

- **Binario:** b'011' -> 0b011
- **Octal:** o'740' -> 0o740
- **Hexadecimal:** z'A3F' -> 0xA3F
- **Lógicos:** .TRUE. -> 1 y .FALSE. -> 0

5.3. Traducción de declaraciones de variables

Las declaraciones de variables se traducen agrupando aquellas que comparten el mismo tipo en una única sentencia de C. Para las variables de tipo **CHARACTER** que especifican una longitud, la dimensión se traslada entre corchetes inmediatamente después del identificador de la variable.

5.4. Traducción de subprogramas

Prototipos (INTERFACE)

Los procedimientos (SUBROUTINE) se traducen como funciones cuyo tipo de retorno es void, mientras que las funciones (FUNCTION) conservan su tipo de datos correspondiente. En el caso de los parámetros de tipo CHARACTER, se generan exclusivamente los corchetes vacíos [] omitiendo la dimensión, en estricto cumplimiento con las especificaciones del enunciado.

Implementaciones

Para garantizar que la traducción de las sentencias interiores sea correcta, se emplean acciones intermedias (*mid-rule actions*) en ANTLR. El ámbito del subprograma se activa inmediatamente antes de parsear la lista de sentencias (sentlist), lo que asegura que:

- En las funciones, cualquier asignación cuyo identificador coincida con el nombre de la propia función se traduzca como return expresion;
- En los procedimientos, los accesos a parámetros declarados como OUT/INOUT incorporen de forma automática el operador de indirección * tanto para operaciones de lectura como de escritura.

5.5. Traducción del programa principal

Las instrucciones correspondientes al bloque del programa principal se sitúan dentro de una función con cabecera *void main(void) { ... }* en el lenguaje objeto. Asimismo, todas las variables locales asociadas al cuerpo principal se declaran de manera estricta al inicio de dicha función *main*.

5.6. Traducción de sentencias

- **Asignaciones:** Se traducen de forma directa debido a la equivalencia sintáctica entre ambos lenguajes. Si el identificador izquierdo equivale al nombre de la función en curso, se traduce como una sentencia return. Si se trata de un parámetro OUT/INOUT, se le antepone el operador *.

- **Llamadas a procedimientos:** La instrucción *CALL nombre(args)*; se traduce eliminando la palabra clave: *nombre(args)*; Si el procedimiento no recibe parámetros, se generan los paréntesis vacíos *nombre()*; Los argumentos asociados a parámetros OUT/INOUT se traducen anteponiendo el operador de dirección **&**.

5.7. Traducción de sentencias de control de flujo

Operadores lógicos y relacionales

Se realiza el mapeo de los operadores lógicos y de comparación hacia sus equivalentes semánticos en C:

- .OR. -> ||
- .AND. -> &&
- .NOT. -> !
- .EQV. -> !^
- .NEQV. -> ^ (XOR)
- /= -> !=

Estructuras de control

- IF: Las tres variantes gramaticales (simple, THEN-ENDIF y THEN-ELSE-ENDIF) se traducen a bloques estandarizados `if(cond){ ... }` e `if(cond){ ... } else { ... }`.
- DO WHILE: Se traduce de manera directa a un bucle `while(cond){ ... }`.
- DO (bucle para): Se transforma en una estructura iterativa `for(vble = ini; vble != lim; vble = vble + inc){ ... }`.
- SELECT CASE: Se mapea mediante una estructura `switch(exp){ case ....: ... break; }`. Cada bloque case incluye la sentencia `break`; al final de su lista de instrucciones, a excepción de la cláusula `DEFAULT` que carece de ella de acuerdo con las especificaciones técnicas solicitadas.

5.8. Traducción de parámetros OUT/INOUT

Los parámetros bajo las directivas OUT e INOUT implementan el paso por referencia simulado mediante punteros en el lenguaje destino C:

- **En la cabecera e interfaz:** Se declaran explícitamente incorporando el operador de puntero (ej. float *c).
- **En el cuerpo de la implementación:** El acceso a su contenido (lectura o modificación) se realiza aplicando la indirección mediante el operador de asterisco (*nombre).
- **En los puntos de llamada:** Se pasa la dirección de memoria de la variable correspondiente utilizando el operador de referencia &nombre.

5.9. Comprobación de nombres en funciones y procedimientos

El analizador semántico realiza tres validaciones de identificadores para asegurar la integridad estructural del código:

1. **Coincidencia de inicio y fin:** El identificador que acompaña a las palabras clave SUBROUTINE o FUNCTION debe coincidir exactamente con el nombre especificado tras la cláusula de cierre END SUBROUTINE o END FUNCTION, aplicándose tanto a bloques de interfaz como a las definiciones completas.
2. **Coincidencia de parámetros:** Los identificadores de la lista de parámetros formales de la cabecera deben coincidir con los nombres utilizados en las especificaciones de tipo dentro de dec_s_paramlist o dec_f_paramlist.
3. **Nombre de retorno en funciones:** El identificador utilizado en la declaración del tipo de retorno de una función debe coincidir con el nombre asignado a la propia función.

El incumplimiento de cualquiera de estas reglas semánticas genera una notificación de error y detiene inmediatamente la escritura del archivo .c de salida.

5.10. Indentación del código generado

El código fuente en el lenguaje objeto se escribe aplicando reglas estrictas de tabulación. Todas las sentencias comprendidas entre los delimitadores de bloque ({ y }) incorporan un nivel adicional de sangrado (4 espacios) respecto a sus líneas de apertura y cierre, asegurando un formato final limpio y legible.

6. Explicación de los Casos de Prueba

Para validar el correcto funcionamiento del analizador léxico y sintáctico, se han diseñado 9 casos de prueba (4 correctos y 5 erróneos) que ponen al límite las restricciones de la gramática. En los casos de prueba correctos se explicará brevemente la parte de la gramática que está probando junto a su árbol sintáctico completo sin errores, mientras que en los casos incorrectos a parte de lo anterior se mostrará el/los errores junto a una breve justificación del error.

Cabe recalcar que, los mensajes de error estarán en el formato de la notificación de error implementada en el apartado anterior(4.1).

La visualización y verificación de los árboles sintácticos generados se ha implementado mediante la herramienta ANTLR Tool Preview que ofrece IntelliJ donde al poner en la terminal o importar un script del caso de prueba se genera el árbol sintáctico completo, indicando el nombre de la regla y el valor que tiene en función del caso de prueba pasado y marcado en rojo los errores. Las capturas de los árboles que se muestran a continuación han sido obtenidas con esta herramienta, todo ello para una mejor ilustración y sencillez a la hora de visualizar esta memoria.

En el proyecto se adjuntan los 9 archivos de casos de prueba utilizados en este trabajo en la carpeta PRUEBAS.

6.1. Casos de Prueba Correctos

Caso Correcto 1- Estructura básica, prueba el esqueleto del programa, la declaración de constantes y variables (con inicialización), y el uso del IF simple de una sola línea validando que acepta sentencias generales y no solo asignaciones.

-Código:

```
PROGRAM test_parte_basico;

INTEGER :: x, y = 10;

REAL, PARAMETER :: PI = 3.141592;

INTERFACE

END INTERFACE

x = 5;

IF (x < y) x = y;

END PROGRAM test_parte_basico
```

-Traducción generada:

```
#define PI 3.141592

void main(void) {

    int x, y = 10;

    x = 5;

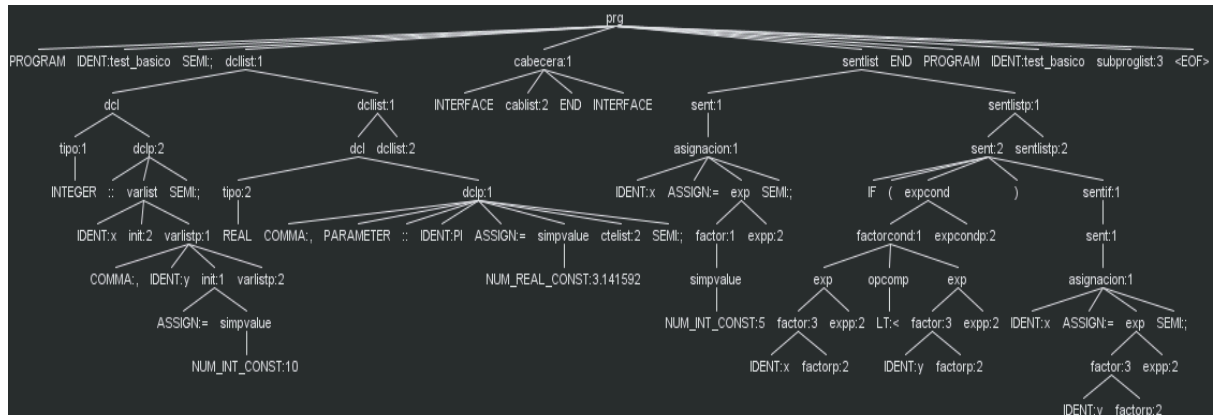
    if(x<y) {

        x = y;

    }

}
```

-Árbol Sintáctico:



Caso Correcto 2- Sentencias de control y parte opcional (bases y condicionales). Verifica las sentencias DO WHILE, SELECT CASE y el uso correcto de las constantes en nuevas bases (como la hexadecimal z'...' y binaria b'...') además de los valores lógicos.

-Código:

```
PROGRAM test_parte_opcional;

INTEGER :: mascara;

CHARACTER(10) :: estado;

INTERFACE

END INTERFACE

mascara = z'A3F';

DO WHILE (.TRUE. .AND. (mascara /= 0))

    SELECT CASE (mascara)

        CASE (b'1010')
```

```

        estado = "ACTIVO";

CASE DEFAULT

        mascara = 0;

END SELECT

ENDDO

END PROGRAM test_parte_opcional

```

-Traducción generada:

```

void main(void) {

    int mascara;

    char estado[10];

    mascara = 0xA3F;

    while(1&&(mascara!=0)) {

        switch(mascara) {

            case 0b1010:

                estado = "ACTIVO";

                break;

            default:

                mascara = 0;

        }

    }

}

```


[illegible]

-Código:

```
PROGRAM Climatizador;

INTEGER :: temp_actual;

INTERFACE

    SUBROUTINE enfriar_rapido

    END SUBROUTINE enfriar_rapido

    SUBROUTINE set_temp(t)

        INTEGER, INTENT(IN) t;
```

```
END SUBROUTINE set_temp
```

```
SUBROUTINE activar_compresor
```

```
END SUBROUTINE activar_compresor
```

```
END INTERFACE
```

```
temp_actual = 30;
```

```
IF (temp_actual > 25) CALL enfriar_rapido;
```

```
CALL set_temp(22);
```

```
END PROGRAM Climatizador
```

```
SUBROUTINE enfriar_rapido
```

```
CALL activar_compresor;
```

```
END SUBROUTINE enfriar_rapido
```

```
SUBROUTINE activar_compresor
```

```
INTEGER :: dummy;
```

```
dummy = 0;
```

```
END SUBROUTINE activar_compresor
```

```

SUBROUTINE set_temp(t)

    INTEGER, INTENT(IN) t;

    temp_actual = t;

END SUBROUTINE set_temp

```

-Traducción generada:

```

void enfriar_rapido(void);

void set_temp(int t);

void activar_compresor(void);

void enfriar_rapido(void) {

    activar_compresor();

}

void activar_compresor(void) {

    int dummy;

    dummy = 0;

}

void set_temp(int t) {

    temp_actual = t;

}

void main(void) {

    int temp_actual;

    temp_actual = 30;

```

```

if(temp_actual>25) {

    enfriar_rapido();

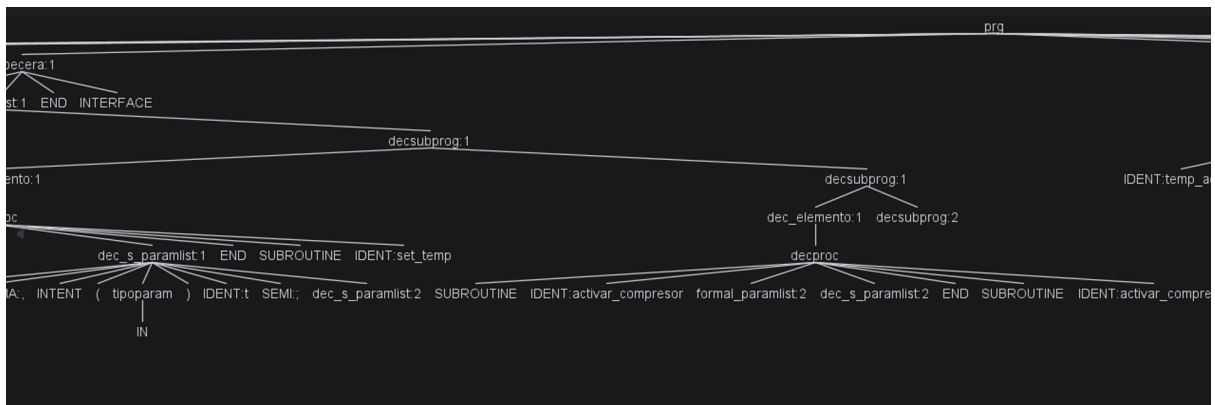
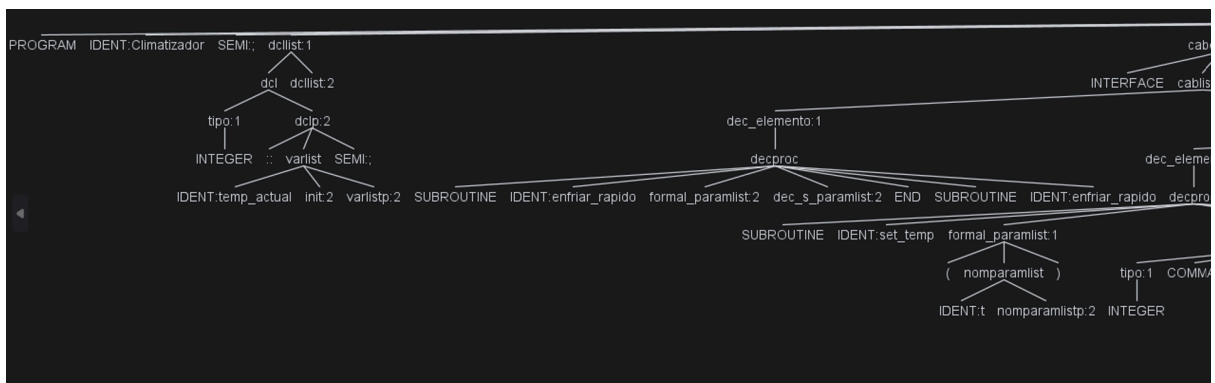
}

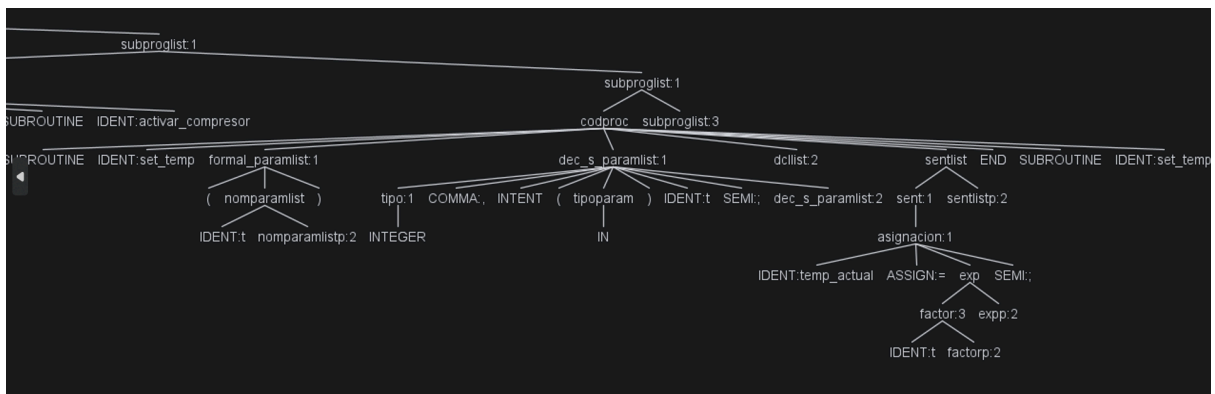
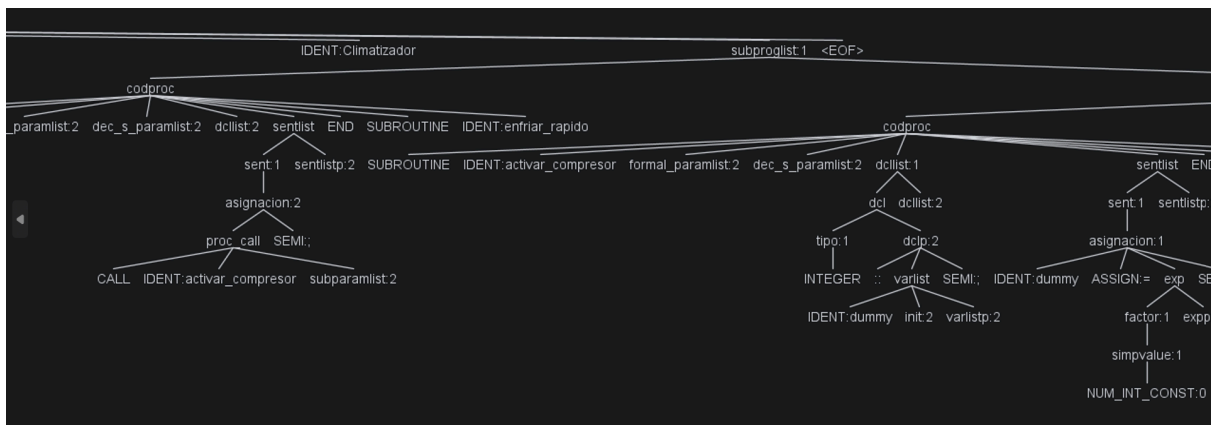
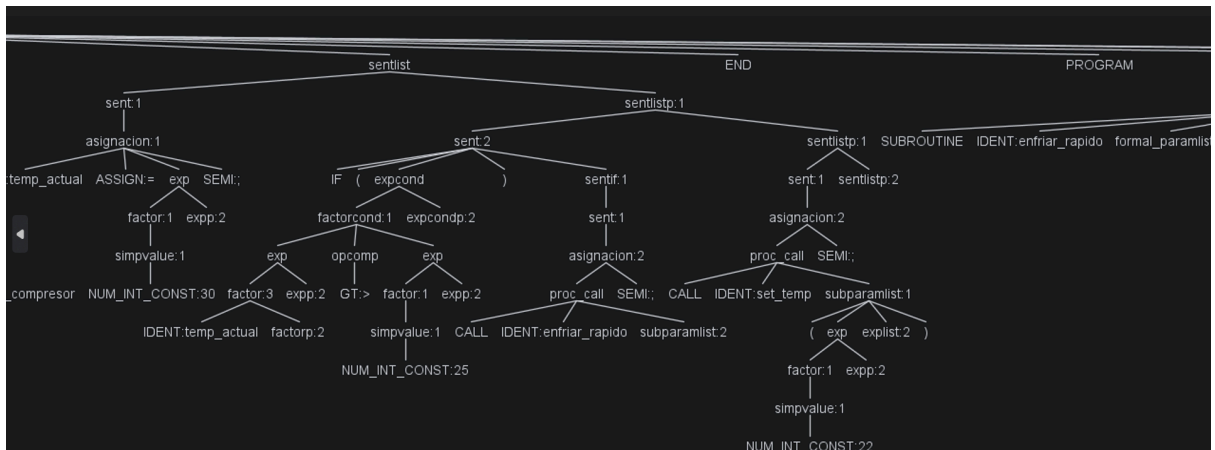
set_temp(22);

}

```

-Árbol Sintáctico:





Caso Correcto 4- Funciones y estructura rígida de retorno. Verifica la estructura estricta de *codfun*, asegurando que el parser procesa correctamente la regla donde la última sentencia antes de END FUNCTION es obligatoriamente la asignación del valor de retorno.

-Código:

```
PROGRAM Conversor;
```

```

REAL :: metros, pies;

INTERFACE

    FUNCTION to_feet(m) REAL :: to_feet;

        REAL, INTENT(IN) m;

    END FUNCTION to_feet

END INTERFACE

metros = 10.5;

pies = to_feet(metros);

END PROGRAM Conversor

FUNCTION to_feet(m) REAL :: to_feet;

    REAL, INTENT(IN) m;

    REAL :: factor;

    factor = 3.2808;

    to_feet = m * factor;

END FUNCTION to_feet

```

-Traducción generada:

```

float to_feet(float m);

float to_feet(float m) {

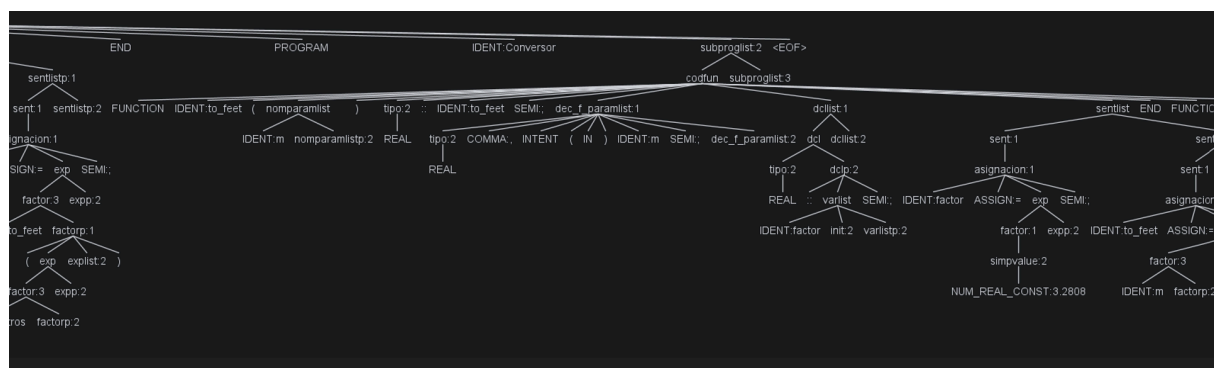
    float to_feet, factor;

    factor = 3.2808;

    return m*factor;
}

```

```
void main(void) {  
  
    float metros, pies;  
  
    metros = 10.5;  
  
    pies = to_feet(metros);  
  
}
```



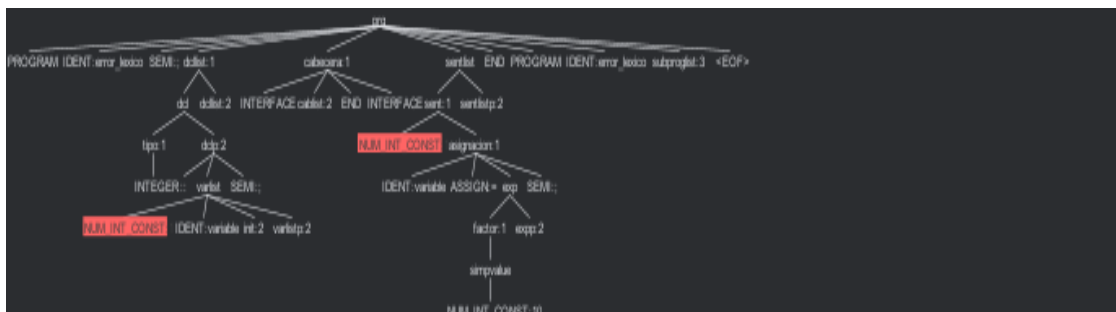
6.2. Casos de Prueba Erróneos y Justificación de Errores

Caso Erróneo 1- Identificador inválido: Intenta declarar una variable iniciada por un dígito (*1_variable*). Es rechazado por el analizador léxico dado que la regla IDENT obliga a que todos los identificadores comiencen exclusivamente por caracteres alfabéticos.

-Código:

```
PROGRAM error_lexico;  
  
INTEGER :: 1_variable;  
  
INTERFACE  
  
END INTERFACE  
  
1_variable = 10;  
  
END PROGRAM error_lexico
```

-Árbol Sintáctico:



-Error:

```
Analizando archivo: PRUEBAS/CasoIncorrecto1.for...
ERROR LÉXICO en línea 3:12
token recognition error at: '_'
ERROR SINTÁCTICO en línea 3:11
extraneous input '1' expecting IDENT
ERROR LÉXICO en línea 6:1
token recognition error at: '_'
ERROR SINTÁCTICO en línea 6:0
extraneous input '1' expecting {'IF', 'DO', 'SELECT', 'CALL', IDENT}
Se detectaron errores. No se generará el archivo C.
```

Los mensajes token recognition error at: '_' confirman que el analizador léxico no reconoce el guión bajo como inicio de token en esa posición, y el extraneous input '1' indica que el dígito es completamente inesperado donde la gramática exige un identificador válido.

Caso Erróneo 2- Lista de sentencias vacía: Presenta un bloque THEN de una sentencia IF sin cuerpo. La regla *sentlist* ha sido diseñada para no permitir cadenas vacías directas, exigiendo sintácticamente al menos una instrucción válida (*sent*) en su interior.

-Código:

```
PROGRAM error_sintactico;

INTEGER :: x = 0;

INTERFACE

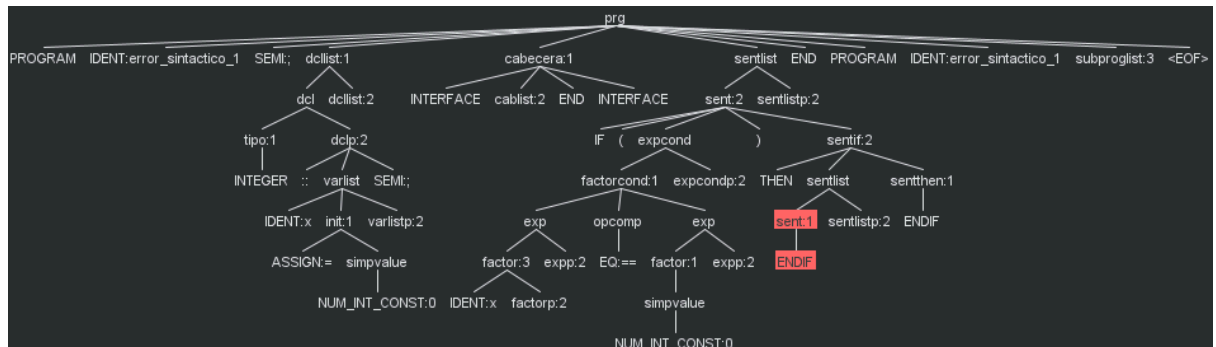
END INTERFACE

IF (x == 0) THEN

ENDIF

END PROGRAM error_sintactico
```

-Árbol Sintáctico:



-Error:

```
Analizando archivo: PRUEBAS/CasoIncorrecto2.for...
ERROR SINTÁCTICO en línea 7:0
mismatched input 'ENDIF' expecting {'IF', 'DO', 'SELECT', 'CALL', IDENT}
Se detectaron errores. No se generará el archivo C.
```

El mensaje (mismatched input 'ENDIF') confirma que el parser esperaba al menos una sentencia válida antes de cerrar el bloque.

Caso Erróneo 3- Ruptura de la jerarquía de bloques (IF anidado): El código presenta una estructura de control IF-THEN anidada donde se intenta cerrar el bloque externo antes que el interno, o se introduce una sentencia en un lugar donde el parser, por su naturaleza LL(1), ya ha "decidido" que el bloque ha terminado.

-Código:

```
PROGRAM error_anidamiento;
```

```
INTEGER :: a = 10, b = 20;
```

```
INTERFACE
```

```
END INTERFACE
```

```
IF (a > 0) THEN
```

IF ($b > 0$) *THEN*

$$a = a + 1;$$

ENDIF

ELSE

$$a = 0;$$

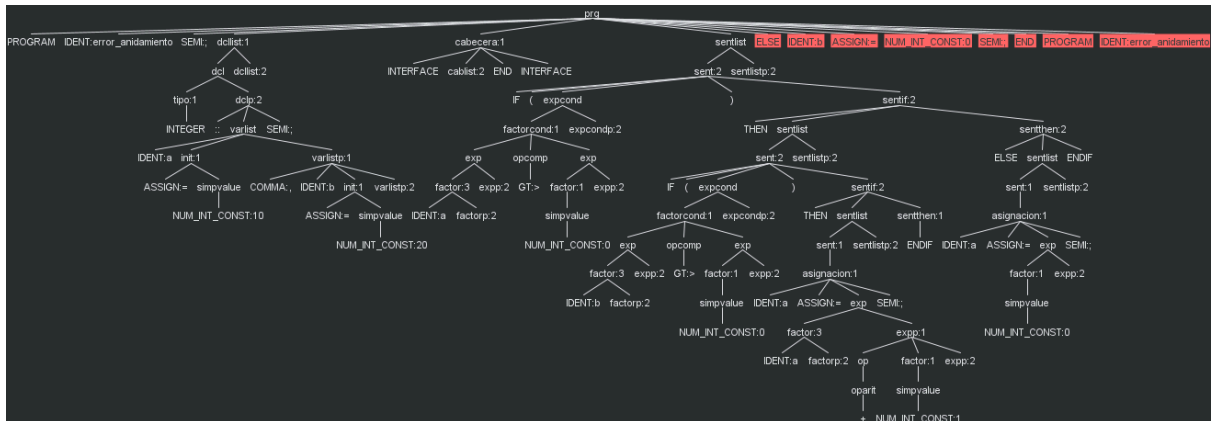
ENDIF

ELSE

$$b = 0;$$

END PROGRAM error_anidamiento

-Árbol Sintáctico:



-Error:

```
Analizando archivo: PRUEBAS/CasoIncorrecto3.for...  
ERROR SINTÁCTICO en línea 14:0  
mismatched input 'ELSE' expecting {'+', '-', '*', '/', ';'}  
Se detectaron errores. No se generará el archivo C.
```

El mensaje mismatched input 'ELSE' confirma que el parser ya no está dentro de una estructura condicional y espera o una nueva sentencia o el final del programa (END PROGRAM), pero nunca un ELSE huérfano.

Caso Erróneo 4 - Ausencia de retorno estricto: La función no sitúa la asignación final de su propio valor (*IDENT = exp ;*) justo antes de la cláusula END FUNCTION. La presencia de otra sentencia (*CALL log_error;*) colocada de manera posterior rompe directamente la estructura de la producción *codfun*.

- Código:

```
PROGRAM error_estructural;

INTEGER :: res;

INTERFACE

    FUNCTION operar (a) INTEGER :: operar;

    INTEGER, INTENT(IN) a;

    END FUNCTION operar

END INTERFACE

res = operar(5);

END PROGRAM error_estructural

FUNCTION operar (a) INTEGER :: operar;

INTEGER, INTENT(IN) a;

IF (a > 0) THEN

    operar = a * 2;

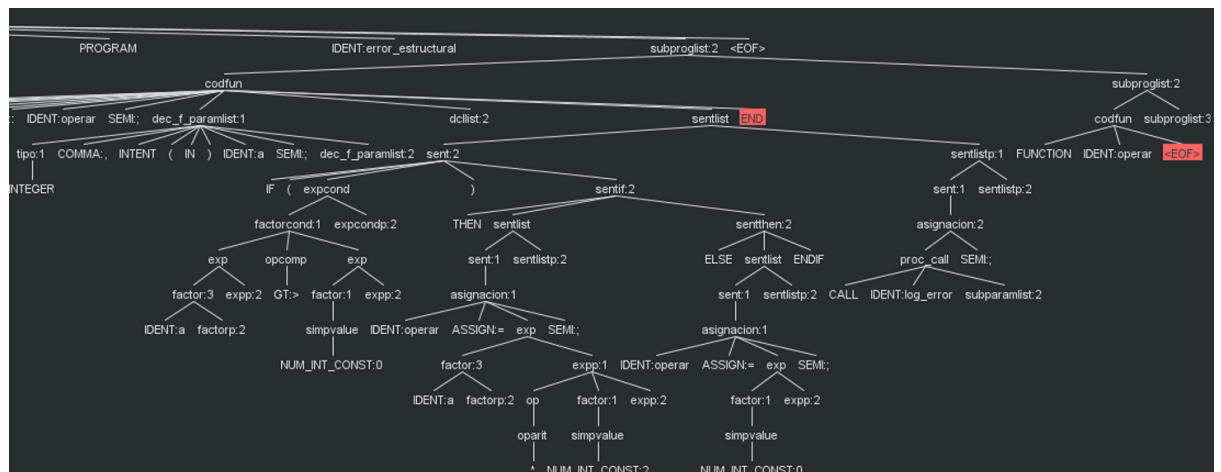
ELSE

    operar = 0;

ENDIF

CALL log_error;

END FUNCTION operar
```

[illegible]

```
Analizando archivo: PRUEBAS/CasoIncorrecto4.for...
ERROR SEMÁNTICO: Nombre de subprograma no coincide en implementación FUNCTION: operar vs operarMAL
Se detectaron errores. No se generará el archivo C.
```

Caso Erróneo 5 - Llamada a procedimiento incorrecta: Se realiza una llamada a un procedimiento con un número de argumentos incorrecto y se pasa una expresión aritmética a un parámetro declarado como OUT/INOUT, cuando este solo admite variables simples. Ambos errores son detectados por el analizador semántico.

- Código:

```
PROGRAM errores_mixtos;

INTEGER :: x = 10;

REAL :: y = 2.5;

INTERFACE

    SUBROUTINE procesar(a, b)

        INTEGER, INTENT(IN) a;

        REAL, INTENT(OUT) b;

    END SUBROUTINE procesar

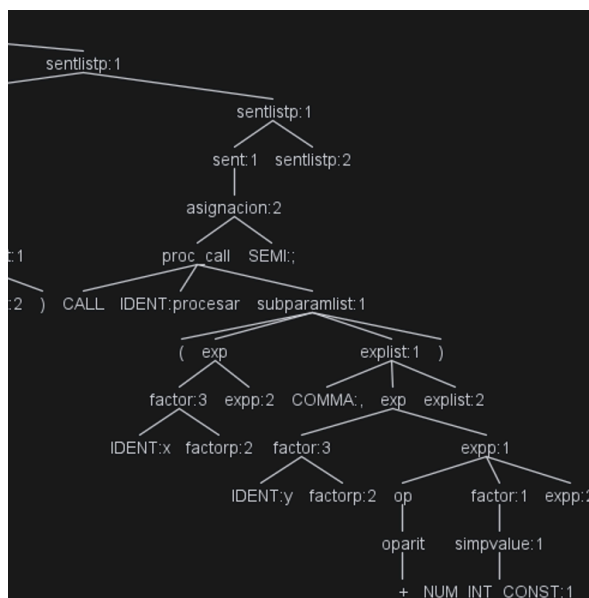
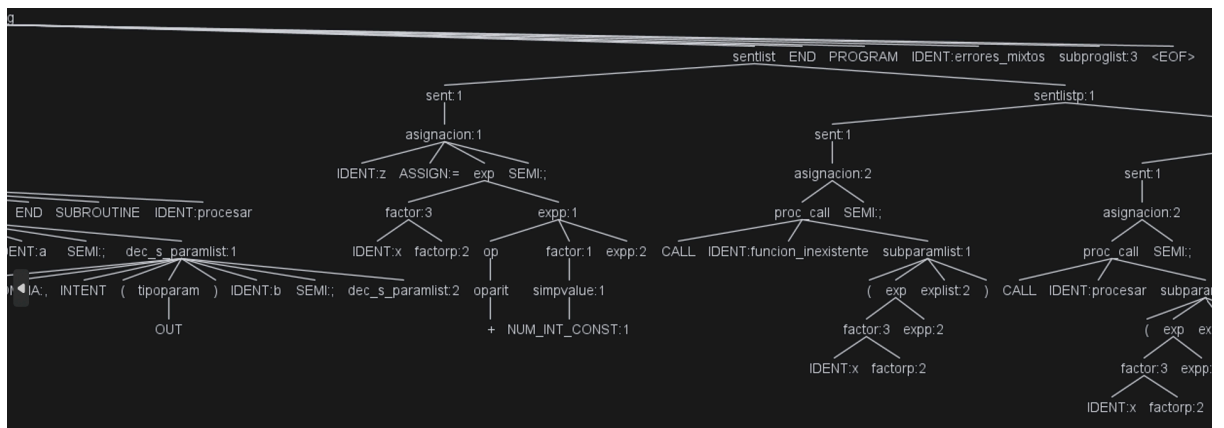
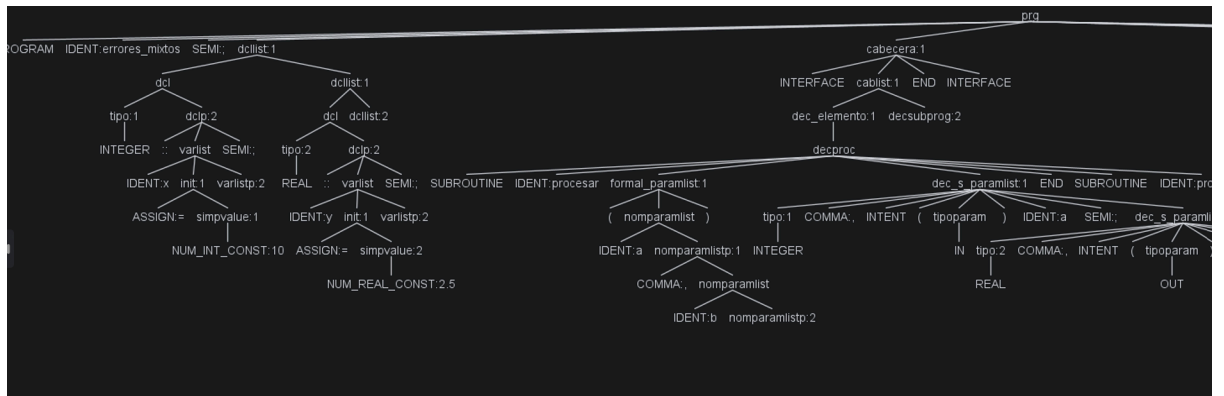
END INTERFACE

CALL procesar(x);

CALL procesar(x, y+1);

END PROGRAM errores_mixtos
```

-Árbol Sintáctico: Al consistir este caso en solo errores semánticos, el árbol sintáctico no mostrará errores:



-Error:

```
Analizando archivo: PRUEBAS/CasoIncorrecto5.for...  
ERROR SEMÁNTICO: Número incorrecto de argumentos en llamada a procesar: se esperaban 2, se recibieron 1  
ERROR SEMÁNTICO: El parámetro 'b' de 'procesar' es OUT/INOUT y debe recibir una variable simple, no una expresión: 'y+1'  
Se detectaron errores. No se generará el archivo C.
```

Los mensajes "Número incorrecto de argumentos en llamada a procesar: se esperaban 2, se recibieron 1" y "El parámetro 'b' de 'procesar' es OUT/INOUT y debe recibir una variable simple, no una expresión: 'y+1'" indican que el analizador semántico detectó dos infracciones en la misma llamada: se omitió un argumento y el parámetro por referencia recibió una expresión aritmética en lugar de una variable simple.

7. Anexo: Conjuntos Directores

A continuación se presentan los conjuntos directores:

```
DIR(prg -> 'PROGRAM' IDENT SEMI dcclist cabecera sentlist 'END'
'PROGRAM' IDENT subproglis EOF) = { PROGRAM }

DIR(dcclist -> dcl dcclist) = { INTEGER, REAL, CHARACTER }

DIR(dcclist -> lambda) = { INTERFACE, IDENT, CALL, IF, DO, SELECT
}

DIR(dcl -> tipo dclp) = { INTEGER, REAL, CHARACTER }

DIR(dclp -> COMMA 'PARAMETER' '::' IDENT ASSIGN simpvalue ctelist
SEMI) = { COMMA }

DIR(dclp -> '::' varlist SEMI) = { :: }

DIR(tipo -> 'INTEGER') = { INTEGER }

DIR(tipo -> 'REAL') = { REAL }

DIR(tipo -> 'CHARACTER' charlength) = { CHARACTER }

DIR(charlength -> '(' NUM_INT_CONST ')') = { ( }

DIR(charlength -> lambda) = { COMMA, ::, INTENT }

DIR(varlist -> IDENT init varlistp) = { IDENT }

DIR(varlistp -> COMMA IDENT init varlistp) = { COMMA }

DIR(varlistp -> lambda) = { SEMI }

DIR(init -> ASSIGN simpvalue) = { ASSIGN }

DIR(init -> lambda) = { COMMA, SEMI }

DIR(ctelist -> COMMA IDENT ASSIGN simpvalue ctelist) = { COMMA }

DIR(ctelist -> lambda) = { SEMI }

DIR(simpvalue -> NUM_INT_CONST | NUM_REAL_CONST | STRING_CONST |
NUM_INT_CONST_B | NUM_INT_CONST_O | NUM_INT_CONST_H | CONST_BOOL)
= { NUM_INT_CONST, NUM_REAL_CONST, STRING_CONST, NUM_INT_CONST_B,
NUM_INT_CONST_O, NUM_INT_CONST_H, CONST_BOOL }

DIR(cabecera -> 'INTERFACE' cablist 'END' 'INTERFACE') = {
INTERFACE }
```

```

DIR(cabecera -> lambda) = { IDENT, CALL, IF, DO, SELECT }

DIR(cablist -> dec_elemento decsubprog) = { SUBROUTINE, FUNCTION }

DIR(cablist -> lambda) = { END }

DIR(decsubprog -> dec_elemento decsubprog) = { SUBROUTINE,
FUNCTION }

DIR(decsubprog -> lambda) = { END }

DIR(dec_elemento -> decproc) = { SUBROUTINE }

DIR(dec_elemento -> decfun) = { FUNCTION }

DIR(decproc -> 'SUBROUTINE' IDENT formal_paramlist dec_s_paramlist
'END' 'SUBROUTINE' IDENT) = { SUBROUTINE }

DIR(decfun -> 'FUNCTION' IDENT '(' nomparamlist ')' tipo '::'
IDENT SEMI dec_f_paramlist 'END' 'FUNCTION' IDENT) = { FUNCTION }

DIR(formal_paramlist -> '(' nomparamlist ')') = { ( }

DIR(formal_paramlist -> lambda) = { INTEGER, REAL, CHARACTER,
IDENT, CALL, IF, DO, SELECT }

DIR(nomparamlist -> IDENT nomparamlistp) = { IDENT }

DIR(nomparamlistp -> COMMA nomparamlist) = { COMMA }

DIR(nomparamlistp -> lambda) = { ) }

DIR(dec_s_paramlist -> tipo COMMA 'INTENT' '(' tipoparam ')' IDENT
SEMI dec_s_paramlist) = { INTEGER, REAL, CHARACTER }

DIR(dec_s_paramlist -> lambda) = { INTEGER, REAL, CHARACTER,
IDENT, CALL, IF, DO, SELECT }

DIR(dec_f_paramlist -> tipo COMMA 'INTENT' '(' 'IN' ')') IDENT SEMI
dec_f_paramlist) = { INTEGER, REAL, CHARACTER }

DIR(dec_f_paramlist -> lambda) = { INTEGER, REAL, CHARACTER,
IDENT, CALL, IF, DO, SELECT }

DIR(tipoparam -> 'IN' | 'OUT' | 'INOUT') = { IN, OUT, INOUT }

DIR(sentlist -> sent sentlistp) = { IDENT, CALL, IF, DO, SELECT }

DIR(sentlistp -> sent sentlistp) = { IDENT, CALL, IF, DO, SELECT }

DIR(sentlistp -> lambda) = { END, ENDIF, ELSE, ENDDO, CASE,
DEFAULT, IDENT }

```

```

DIR(sent -> asignacion) = { IDENT, CALL }

DIR(sent -> 'IF' '(' expcond ')' sentif) = { IF }

DIR(sent -> 'DO' sentdo) = { DO }

DIR(sent -> 'SELECT' 'CASE' '(' exp ')' casos 'END' 'SELECT') = {
SELECT }

DIR(asignacion -> IDENT ASSIGN exp SEMI) = { IDENT }

DIR(asignacion -> proc_call SEMI) = { CALL }

DIR(sentif -> sent) = { IDENT, CALL, IF, DO, SELECT }

DIR(sentif -> 'THEN' sentlist sentthen) = { THEN }

DIR(sentthen -> 'ENDIF') = { ENDIF }

DIR(sentthen -> 'ELSE' sentlist 'ENDIF') = { ELSE }

DIR(sentdo -> 'WHILE' '(' expcond ')' sentlist 'ENDDO') = { WHILE
}

DIR(sentdo -> IDENT ASSIGN doval COMMA doval COMMA doval sentlist
'ENDDO') = { IDENT }

DIR(doval -> NUM_INT_CONST) = { NUM_INT_CONST }

DIR(doval -> IDENT) = { IDENT }

DIR(casos -> 'CASE' casosp) = { CASE }

DIR(casos -> lambda) = { END }

DIR(casosp -> '(' etiquetas ')' sentlist casos) = { ( }

DIR(casosp -> 'DEFAULT' sentlist) = { DEFAULT }

DIR(etiquetas -> simpvalue etiquetasp) = { NUM_INT_CONST,
NUM_REAL_CONST, STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O,
NUM_INT_CONST_H, CONST_BOOL }

DIR(etiquetas -> COLON simpvalue) = { COLON }

DIR(etiquetasp -> listaetiquetas) = { COMMA, ) }

DIR(etiquetasp -> COLON simpvaluep) = { COLON }

DIR(listaetiquetas -> COMMA simpvalue listaetiquetas) = { COMMA }

DIR(listaetiquetas -> lambda) = { ) }

```

```

DIR(simpvaluep -> simpvalue) = { NUM_INT_CONST, NUM_REAL_CONST,
STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O, NUM_INT_CONST_H,
CONST_BOOL }

DIR(simpvaluep -> lambda) = { }

DIR(proc_call -> CALL IDENT subparamlist) = { CALL }

DIR(subparamlist -> '(' exp explist ')') = { ( }

DIR(subparamlist -> lambda) = { SEMI }

DIR(explist -> COMMA exp explist) = { COMMA }

DIR(explist -> lambda) = { ) }

DIR(exp -> factor exp) = { NUM_INT_CONST, NUM_REAL_CONST,
STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O, NUM_INT_CONST_H,
CONST_BOOL, (, IDENT }

DIR(exp -> op factor exp) = { +, -, *, / }

DIR(exp -> lambda) = { SEMI, ), COMMA, THEN, ELSE, ENDIF, ENDDO,
CASE, DEFAULT, END, LT, GT, LE, GE, EQ, NE, .OR., .AND., .EQV.,
.NEQV. }

DIR(op -> oparit) = { +, -, *, / }

DIR(oparit -> '+' | '-' | '*' | '/') = { +, -, *, / }

DIR(factor -> simpvalue) = { NUM_INT_CONST, NUM_REAL_CONST,
STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O, NUM_INT_CONST_H,
CONST_BOOL }

DIR(factor -> '(' exp ')') = { ( }

DIR(factor -> IDENT factorp) = { IDENT }

DIR(factorp -> '(' exp explist ')') = { ( }

DIR(factorp -> lambda) = { +, -, *, /, SEMI, ), COMMA, THEN, ELSE,
ENDIF, ENDDO, CASE, DEFAULT, END, LT, GT, LE, GE, EQ, NE, .OR.,
.AND., .EQV., .NEQV. }

DIR(expcond -> factorcond expcondp) = { NUM_INT_CONST,
NUM_REAL_CONST, STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O,
NUM_INT_CONST_H, CONST_BOOL, (, IDENT, .NOT. }

DIR(expcondp -> oplog factorcond expcondp) = { .OR., .AND., .EQV.,
.NEQV. }

```

```

DIR(expcondp -> lambda) = { }, THEN, ELSE, ENDIF, ENDDO }

DIR(factorcond -> exp opcomp exp) = { NUM_INT_CONST,
NUM_REAL_CONST, STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O,
NUM_INT_CONST_H, CONST_BOOL, (, IDENT }

DIR(factorcond -> '(' expcond ')') = { ( }

DIR(factorcond -> '.NOT.' factorcond) = { .NOT. }

DIR(factorcond -> CONST_BOOL) = { CONST_BOOL }

DIR(oplog -> '.OR.' | '.AND.' | '.EQV.' | '.NEQV.') = { .OR.,
.AND., .EQV., .NEQV. }

DIR(opcomp -> LT | GT | LE | GE | EQ | NE) = { LT, GT, LE, GE, EQ,
NE }

DIR(subproglis -> codproc subproglis) = { SUBROUTINE }

DIR(subproglis -> codfun subproglis) = { FUNCTION }

DIR(subproglis -> lambda) = { EOF }

DIR(codproc -> 'SUBROUTINE' IDENT formal_paramlist dec_s_paramlist
dcclist sentlist 'END' 'SUBROUTINE' IDENT) = { SUBROUTINE }

DIR(codfun -> 'FUNCTION' IDENT '(' nomparamlist ')' tipo '::'
IDENT SEMI dec_f_paramlist dcclist sentlist IDENT ASSIGN exp SEMI
'END' 'FUNCTION' IDENT) = { FUNCTION }

```